

KidView

Product Requirements Document

Version 1.0 - Initial Build Scope

Purpose: a calm, parent-shaped YouTube discovery gateway for children

Field	Value
Project name	KidView
Primary platform	Responsive web app, tablet-first
Initial source library	YouTube only
Initial users	One household, one or more child profiles
Future design target	Multi-household support without major rewrite
Technical baseline	Node.js, Express, EJS, vanilla JS, SQLite, OpenLiteSpeed, EC2

1. Product Summary

KidView is a family-controlled video discovery and playback web app designed to let children search for and watch online video content with guardrails that reduce exposure to unsafe, manipulative, addictive, or low-quality platform behavior.

The product is not intended to replace parenting, guarantee perfect safety, or create a fully sanitized version of YouTube. Its goal is narrower and more realistic: provide a calmer, search-based, parent-shaped video discovery layer that lets a child explore interests with limited independence while avoiding the open-ended recommendation loops, thumbnails, comments, autoplay, Shorts, livestreams, and algorithmic engagement mechanics of major video platforms.

The initial version will focus only on YouTube-sourced videos. It will not provide general web search, Spotify browsing, TikTok, Instagram, social media access, messaging, comments, live video access, or native YouTube browsing.

2. Problem Statement

Children around age 10 are increasingly curious about media, music, movies, creators, behind-the-scenes content, hobbies, and popular culture. A complete block on web/video access may become unrealistic or counterproductive, but unrestricted access to YouTube, Google, Spotify video, social media, and algorithmic recommendation feeds introduces several problems.

Harmless initial searches can lead to algorithmically recommended content that is lower-quality, more addictive, more consumerist, more mature, or more manipulative. Existing child-oriented products may be too limited, too chaotic, or poorly aligned with the family's values and the child's actual interests.

KidView aims to create a healthier middle path: controlled discovery instead of either total prohibition or full platform access.

3. Product Philosophy

KidView should be calm, deliberate, and boring by design. The app should help a child answer: 'What do I want to learn about or watch right now?' It should not ask: 'What can we show next to keep this child watching?'

The app should help the child articulate search intent without silently taking over the query. It should return very few choices, hide engagement-optimized visual bait, avoid autoplay and infinite feeds, and let parents shape policies through video-level and channel-level decisions.

The product is not 'safe YouTube.' It is a guided discovery layer that uses YouTube as a source library while reducing exposure to YouTube's addictive interface patterns.

4. Target Users

4.1 Primary Child User

A child roughly ages 8-12 who wants to search for and watch videos related to interests such as movies, music, science, nature, animals, books, crafts, sports, cooking, hobbies, and occasional light entertainment.

The child user should not need a real login account. The child can use a selected profile on a trusted household device after a parent has authenticated the device.

4.2 Parent/Admin User

- Configure child profiles and basic policy settings.
- Review searches and watched/selected videos.
- Approve or block videos.
- Approve, block, or flag channels for review.
- See why a video was allowed, blocked, or queued for review.
- Override model decisions.

4.3 Future User Base

KidView may later support friends and family outside the original household. Therefore, the data model should support multiple households from the beginning, even if v1 has only one household in practice.

5. Goals and Non-Goals

5.1 MVP Goals

- Allow a child to search for YouTube videos through a controlled web interface.
- Return either one clarification prompt or up to three approved video results.
- Use standardized category icons instead of native YouTube thumbnails.
- Allow one embedded video to play at a time.
- Block access to comments, infinite scroll, autoplay, Shorts, livestreams, native YouTube browsing, and algorithmic recommendation feeds.
- Let the child use simple feedback actions: not what I meant, more like this, this feels weird.
- Let the parent approve/block videos and approve/block/review-first channels.

- Use LLM-assisted query clarification and moderation with structured outputs.
- Use transcript sampling selectively for better moderation confidence without storing full transcripts.
- Cache video metadata and moderation reviews.

5.2 Long-Term Goals

- Multiple households and invitations for additional parent/admin users.
- Managed authentication, if friends/family use expands.
- Category-based time limits and scheduling rules.
- Parent-approved topic suggestions and safer more-like-this search generation.
- Optional richer transcript review and improved confidence scoring.
- TV-specific route with remote-friendly focus behavior.
- Migration from SQLite to Postgres if multi-household usage or concurrency grows.

5.3 Explicit Non-Goals for V1

- General web search.
- Social media browsing or messaging.
- Spotify integration.
- TikTok, Instagram, YouTube Shorts, or Shorts-style feeds.
- YouTube livestream access.
- Downloading and locally serving video files.
- Building a full recommender or analytics platform.
- Real-time chat with a child.
- Fully automated trust of unknown channels.
- Showing native YouTube thumbnails in child search results.
- Showing blocked-video reasons to the child.
- Guaranteeing perfect safety.

6. Core Product Principles

Calm discovery

The UI should reduce urgency and stimulation. It should be intentionally calm, with limited choices, large readable cards, muted visual design, and no attention-grabbing thumbnails.

Guided search, not query takeover

The app may help rewrite a child's query, correct spelling, and clarify intent. It should not silently over-transform a query into something the child did not ask for.

Maximum three results

For each search, KidView should return no more than three video results. This is a core product decision, not a temporary UI shortcut.

No native thumbnails

The child-facing UI should never show native YouTube thumbnails. Each result should use a standardized icon selected by KidView based on video category and classification.

No Shorts or livestreams

Short-form and live video formats should be prohibited in v1. These formats are too open-ended, fast-moving, and difficult to moderate reliably in the intended product model.

Parent decisions override model decisions

The AI moderation layer is advisory. The parent is the final authority within a household.

Channel-level decisions matter

Video-level approval is not enough. Parents should be able to mark channels as generally okay, review-first, or blocked.

Child profiles are not full user accounts

A child profile represents preferences, policies, and activity history. It does not need to be a login identity.

Do not create curiosity through block messages

When content is blocked, the child should not be shown detailed risk labels. The child-facing message should be neutral and redirective.

7. MVP User Experience

7.1 Child Search Flow

1. Child opens the child-facing KidView page.
2. Child enters a search query.
3. Server evaluates the query for intent and ambiguity.
4. If the query is clear enough, the server searches YouTube, filters/moderates candidates, and returns up to three approved results.
5. If the query is broad or ambiguous, KidView asks one clarification question with a few choice chips.
6. If no suitable results are found, KidView gives a neutral message and offers safer/refined search suggestions.

Example: original query "harry potter hermione" may generate clarification chips such as behind-the-scenes, interviews with Emma Watson, movie clips, costumes and props.

7.2 Search Result UI

- Standardized category icon.
- Clean title.
- Channel name.
- Video length.
- One-sentence why this matched explanation.
- One or two child-safe labels.
- Watch button.
- Not what I meant button.
- No native YouTube thumbnail.

7.3 Video Playback UI

- Embed one YouTube video.
- Use as little native YouTube UI as practical.
- No autoplay.
- No playlists.
- No livestreams.
- No comments.
- No child-facing related-video panel.
- App-owned controls where feasible.
- Calm completion screen after video ends.
- Feedback actions: more like this, not what I meant, this feels weird.

7.4 Parent Dashboard

- Recent searches.
- Recent watched/selected videos.
- Videos needing review.
- Parent-approved and parent-blocked videos.
- Channel decisions.
- Basic child profile settings.
- Moderation explanation view.
- Parent override controls.

8. Search Behavior

8.1 Query Interpretation

KidView should use an LLM to interpret child search queries. This may include spelling correction, intent detection, query cleanup, clarification chip generation, safer search phrasing, and category inference.

The app should not automatically convert every query into an official, educational, or parent-preferred query. Kids often know what they want but do not know how to express it in search syntax. KidView should help them articulate intent, not seize control of the search.

8.2 Clarification Logic

KidView should ask a clarification question when the query is too broad, ambiguous, mixed-risk, or format-unclear. It should ask at most one clarification question before returning results.

Query	Clarification chips
Taylor Swift	Songs; live performances; interviews; songwriting; behind-the-scenes
Benson Boone	Songs; live performances; interviews; behind-the-scenes
Harry Potter	Behind-the-scenes; character interviews; costumes and props; studio tour; movie clips

8.3 Result Strategy

For a successful query, KidView should prefer: best match to child intent, safer or more official version of the same intent, and an adjacent but still relevant option. The app should show fewer than three results if only one or two pass review.

9. Content Taxonomy

KidView should classify videos using both categories and tags. Categories describe what the video is about. Tags describe format, quality signals, and risks. Do not collapse these into one field.

Type	Values
Primary categories	Science & Nature; Animals; History & Culture; Movies & Behind-the-Scenes; Music; Sports & Skills; Art & Making; Books & Stories; Gaming; Toys & Play; Shopping & Unboxing; Fashion & Beauty; Food & Cooking; News & Real World; Comedy; General Creator Video; Unknown
Content tags	Official/source-backed; Educational; Instructional; Interview; Performance; Review; Unboxing; Behind-the-scenes; Documentary-style; Music video; Live performance; Tutorial; Commentary; Skit; Challenge; Product-focused; Sponsored; Creator personality-driven
Risk/quality tags	Mild language; Mature themes; Scary or intense; Sad or upsetting; Violence or injury; Consumerist; Body image/status pressure; Parasocial creator pull; High-energy/attention-grabby; Low-value filler; Misleading/clickbait; Unknown context; Needs parent review

10. Moderation Model

10.1 Moderation Inputs

- Video title, channel title, and description or truncated description.
- Duration, publish date, YouTube category if available, and livestream/Short signals.
- Channel trust status and prior household decisions.
- Transcript sample or transcript summary when available and needed.
- Parent policy settings and search query context.

10.2 Transcript Strategy

Transcript support should be included in v1 as an optional moderation input, not as a requirement for every candidate. KidView should not store full transcripts after a moderation decision is made.

The app should use transcript sampling selectively for unknown, vague, borderline, or higher-risk videos. Sampling should generally include the beginning, middle, and end of the transcript rather than only the first few hundred characters.

- Under roughly 8 minutes: full transcript may be sent if available and within a configured character/token cap.
- 8-30 minutes: send start/middle/end samples under a total configured cap.
- Over 30 minutes: use samples only and lean toward review unless the channel is trusted.
- Over 60 minutes: review by default unless the channel or exact video is already approved.
- If transcript is unavailable, reduce confidence and require review unless other trust signals are strong.

Store transcript metadata only: whether a transcript was available, whether a transcript sample was used, transcript source, approximate character count, optional excerpt hash, and transcript-derived summary. Do not persist full transcript text by default.

10.3 Moderation Output

The moderation service should return structured JSON, not prose-only output. Suggested fields:

```
{
  "primary_category": "Movies & Behind-the-Scenes",
  "secondary_categories": ["Books & Stories"],
  "content_tags": ["Behind-the-scenes", "Interview"],
  "risk_tags": ["Mild fantasy themes"],
  "quality_tags": ["Official/source-backed"],
  "age_fit_score": 4,
  "educational_value_score": 3,
  "interest_match_score": 5,
  "consumerism_score": 1,
  "addictive_design_risk_score": 2,
  "mature_content_risk_score": 1,
  "language_risk_score": 1,
  "fear_or_distress_risk_score": 2,
  "low_value_filler_score": 1,
  "confidence_score": 4,
  "decision": "allow",
  "child_explanation": "This looks like a behind-the-scenes video about the Harry Potter movies.",
  "parent_explanation": "The video appears to match the child search and no obvious mature content was found in available metadata or transcript samples."
}
```

10.4 Decision Values

Decision	Meaning
allow	May be shown to child.
allow_limited	May be shown under future category/time limits.
review	Parent must approve before child sees it.
block	Do not show to child.
unknown	Insufficient confidence; do not show to child.

11. Data Model

The v1 database should stay simple while preserving long-term flexibility. Recommended database: SQLite for v1, with migrations and a schema design that can later move to Postgres.

Table	Purpose
households	Family/account container. Nearly all family-

	specific records should include household_id.
parent_users	Authenticated adult/admin users.
child_profiles	Child profile, policy, and activity history; not necessarily a login account.
policy_profiles	Child-specific or household-specific content settings.
channels	YouTube channel metadata.
videos	YouTube video metadata. Do not store native thumbnails for child-facing use.
moderation_reviews	Model/app review of a video, including category, tags, decision, confidence, prompt version, and whether transcript was used.
household_video_decisions	Parent decisions scoped to a household.
household_channel_decisions	Channel-level trust decisions scoped to a household.
search_events	Original query, clarified query, choices shown, clicked video, and feedback.

11.1 Data Minimization

Store decisions and structured assessments, not everything. Do not store full video files, native thumbnails, full transcripts, large raw API response blobs, detailed second-by-second engagement logs, or unrelated analytics data by default.

12. Technical Architecture

12.1 Existing Infrastructure Assumptions

- AWS EC2 instance running Ubuntu.
- OpenLiteSpeed reverse proxy / virtual host setup.
- Route53 DNS.
- HTTPS via Certbot.
- Node/Express apps running on separate ports/subdomains.
- PM2 for process management.
- GitHub source control.
- VS Code and Codex-assisted development.

12.2 Recommended Stack

Area	Recommendation
Backend	Node.js, Express, EJS templates, vanilla JavaScript.
Database	SQLite for v1, preferably with Drizzle ORM or better-sqlite3 plus migration files.
APIs	YouTube Data API, YouTube IFrame Player API, OpenAI API.
Frontend	Server-rendered EJS, plain CSS variables, minimal client-side JavaScript, static SVG icons.
Hosting	Dedicated subdomain, dedicated Express port, OpenLiteSpeed reverse proxy, HTTPS, PM2, .env

12.3 Suggested Node Packages

- express, ej, dotenv, helmet, express-rate-limit, cookie-parser, express-session.
- connect-sqlite3 or equivalent session store.
- better-sqlite3, optionally drizzle-orm and drizzle-kit.
- zod for validating structured AI/API responses.
- openai for OpenAI API calls.
- googleapis or direct fetch calls to YouTube Data API.
- nanoid or built-in UUID utilities.
- bcrypt if local password auth is used.
- nodemon for development.

12.4 Suggested Folder Structure

```

/app
  /db
    schema.js
    migrations/
    queries/
  /services
    youtubeService.js
    openaiService.js
    moderationService.js
    transcriptService.js
    searchService.js
    policyService.js
  /routes
    childRoutes.js
    parentRoutes.js
    authRoutes.js
    apiRoutes.js
  /views
    /child
    /parent
    /partials
  /public
    /css
    /js
    /icons
  server.js
  config.js

```

13. YouTube Integration

13.1 Search and Metadata

Use the YouTube Data API from the server only. Keep API keys server-side. Request a limited number of candidate results, fetch additional metadata, filter candidates before moderation, and cache reviewed videos.

13.2 Candidate Filtering

- Exclude Shorts where detectable.

- Exclude livestreams and upcoming live broadcasts.
- Exclude videos that are not embeddable.
- Exclude videos from blocked channels.
- Exclude videos with obviously inappropriate titles.
- Exclude excessively long videos unless allowed by policy.
- Prefer known/approved channels.
- Prefer videos with clear metadata and available transcripts when useful.

13.3 Playback

Use embedded YouTube playback, but assume the iframe cannot be fully transformed into a native video element. Configure the player as minimally as possible, avoid autoplay and playlists, hide native recommendations outside the iframe, and replace the player with a calm completion screen when playback ends.

14. Security and User Management

14.1 Authentication

V1 may use simple parent login if limited to one household. If friends/family access becomes likely, use managed authentication rather than building a complex password system manually. Candidate options include Clerk, Auth0, Supabase Auth, Firebase Auth, or Auth.js.

14.2 Authorization

Separate authentication, authorization, and child policy. Every household-specific route should verify that the logged-in parent has access to that household.

14.3 Security Basics

- HTTPS only.
- Secure HTTP-only cookies.
- SameSite cookie settings.
- CSRF protection for state-changing actions.
- Rate limiting.
- Server-side sessions.
- No API keys in browser JavaScript.
- Environment variables for secrets.
- Parent dashboard access restricted and preferably re-authenticated/PIN-gated from child mode.

15. UI and Device Support

The child UI should be tablet-first and touch-friendly. iPad and iPhone should be primary targets. Desktop can be used for the parent dashboard. LG WebOS browser support should be treated as experimental unless tested frequently on the actual TV.

- Large search box and large buttons.
- No hover-only interactions.
- No dense tables in child UI.
- No infinite scroll.

- No native thumbnails.
- Calm colors and clear typography.
- Minimal animation.
- No streaks, badges, or gamification.
- Separate /tv route later if TV support becomes important.

16. MVP Acceptance Criteria

- Parent can log in.
- Parent can create or use a child profile.
- Child can search for a topic.
- App asks a clarification question when appropriate.
- App returns no more than three approved results.
- App never shows native YouTube thumbnails in child search results.
- App prohibits Shorts and livestreams.
- Each result uses a standardized icon.
- Child can watch one embedded video.
- App does not provide comments, infinite scroll, autoplay, or a native recommendation feed.
- Parent can approve or block a video.
- Parent can approve, block, or require review for a channel.
- Model moderation results are stored and visible to the parent.
- Transcript samples may be used for moderation, but full transcripts are not stored after decisioning.
- Parent decisions override model decisions.
- Blocked videos are not shown to the child with curiosity-triggering explanations.
- Video and channel decisions are scoped to a household.
- App runs on the existing Node/Express/OpenLiteSpeed infrastructure.

17. Phased Roadmap

Phase	Scope
Phase 0: Setup	Create repo, Express/EJS skeleton, .env, SQLite, initial schema/migrations, parent login, seed household and child profile.
Phase 1: Basic Search	Child search page, YouTube API connection, candidate metadata storage, icon system, up to three result cards, no thumbnails.
Phase 2: Query Clarification	LLM query interpretation, broad/ambiguous detection, clarification chips, one clarification step.
Phase 3: Moderation	Structured moderation service, transcript sampling when useful, cached reviews, allow/review/block/unknown decisions.
Phase 4: Playback	Embedded player page, minimal player behavior, completion screen, feedback actions.
Phase 5: Parent Dashboard	Review queues, approve/block video, approve/block/review channel, moderation detail view.
Phase 6: Hardening	Auth/session security, rate limits, logging, backup

	strategy, device testing, quota tracking.
Phase 7: Expansion	Multi-household support, managed auth, invites, category policies, Postgres migration, TV route.

18. Recommended V1 Defaults

Policy Area	Default
Allow by default	Science, nature, animals, books, art, making, cooking, official music performances, movie behind-the-scenes.
Review first	General creator videos, shopping/unboxing, fashion/beauty, gaming, comedy, news/current events, unknown channels.
Block by default	Shorts, livestreams, mature themes, violent/injury content, prank/challenge content, unknown low-confidence videos.
Results	Maximum three. Show fewer if only fewer pass review.
Clarification	Maximum one clarification question.
Playback	No autoplay, no playlists, no livestreams, completion screen after video ends.
Recommendations	App-generated search refinements only, never native YouTube recommendations.
Transcript handling	Use samples when useful. Do not store full transcripts after decisions.

19. Open Questions

- Should v1 use Drizzle ORM or raw SQL with better-sqlite3?
- Should parent login be local email/password, Google login, or managed auth?
- Which transcript source/library is acceptable for v1 given reliability and terms-risk tradeoffs?
- Should allow_limited be operational in v1, or stored only for future policy controls?
- What exact categories should be enabled by default for the first child profile?
- Should child mode require parent PIN to exit or just rely on device-level restrictions?
- How strict should the app be with unknown channels?
- What subdomain should be used for production?

20. Implementation Guidance

Keep the codebase boring. Use server-rendered pages, simple services, plain JavaScript, and a small database. Avoid adding frameworks or infrastructure because they seem impressive. The project should be easy to inspect, debug, and modify.

The correct first version is not the smartest possible system. It is the smallest useful loop:

- Child searches.
- App clarifies if needed.
- App finds candidates.

- App filters and moderates, using transcript samples when useful.
- App shows up to three calm choices.
- Child watches one video.
- Parent can review and override.
- The system remembers the decision.